Lapi OCI provisioning and deployment

Final deployment handover for Swamy
PowerShell or OCI Cloud Shell | Branch lapi_sprint1 | 16 October 2026

Follow this guide in two stages: first provision and initialize the OCI destination, then deploy selected database objects, frontend and backend code. The first section includes the recorded Mumbai pilot example and each resource/model purpose. The second covers the AI skill and JSON with either Windows PowerShell or OCI Cloud Shell.

Guide order

Section	What it covers	Pages
1 Resource provisioning	OCI and model prerequisites; networking, OKE, database, storage, registry, initial app setup and handoff.	2-7
2 Code deployment	Prerequisites; automatic Windows setup; saved-file builds; target JSON; approvals; Cloud Shell; verification.	8-18

1 Resource provisioning

For a new OCI destination, complete the one-time platform and initial application setup first. Use the pilot values as an observed example, then confirm the destination names, sizing, permissions, model availability and cost. The platform/application owner prepares the resources and protected configuration before the code release.

2 Code deployment

For an already prepared destination, start on page 8. The workflow diagram is on page 9. Select database objects, frontend and backend independently with Y/N flags. Backend includes API and worker. The AI skill and deploy.ps1 preserve existing infrastructure, application settings, stored data and model registrations. They do not provision an empty tenancy or initialize a bare cluster.

For Windows, read the prerequisites first on pages 8 and 10. Run **Setup-Lapi.ps1**, then **Deploy-Lapi.ps1**. Saved local source is built by default; no commit or Ref update is required. The AI route is on page 14 and OCI Cloud Shell remains on pages 16-17.

```
Set-Location .\lapi_deploy
.\Setup-Lapi.ps1
# After LAPI_PREREQUISITES_READY:
.\Deploy-Lapi.ps1
```

Review the exact target, source, missing DB objects/schema and selected images before publication approval. Existing configuration is preserved; the Recreate strategy can briefly interrupt service when images change. No backup is created or required by the code/object release workflow.

1 Resource provisioning

This section helps the platform/application owner prepare a **new** destination, separately from the code/object release in Section 2. Pilot values were recorded on 15-16 October 2026; verify availability, sizing and cost for the new target. Leave existing live resources unchanged.

OCI prerequisites and purpose

Resource or prerequisite	Purpose	Pilot example and requirement
Tenancy, compartment, region and IAM	Select the authorized target; permit resource management, cluster access, image push/pull and bucket operations.	Required. Mumbai ap-mumbai-1; approve the new compartment, identity, limits and budget before creation.
VCN, subnets, routing and security rules	Connect the API endpoint, workers and public entry; allow required database, OCI and model traffic.	Required. lapi-vcn 10.42.0.0/16; API .0/24, workers .1/24, load balancer .2/24.
OKE cluster and managed node pool	Run frontend, FastAPI, agents, worker, HTTPS and CPU reranker.	Required for this hosting design. Basic lapi-basic; 1 VM.Standard.A1.Flex ARM64 node, 2 OCPUs, 12 GB RAM; observed Kubernetes v1.36.4.
Boot and persistent block storage	Boot the worker; retain protected app state/model assets and certificate state across pod replacements.	Required. 50 GB boot; application-state and https-state claims each request 50 Gi, backed by 50 GB OCI volumes. oci-bv storage class.
Autonomous AI Database and wallet	Store app records, access, durable jobs and Oracle native vectors; authenticate the DB connection.	Required. lapi_ora_db / lapiora, Oracle 26ai, observed 1 ECPU and 20 GB; ADMIN, lapiora_low, mTLS wallet. Not a new-target size default.
OCI Object Storage bucket	Store original documents, prompts, parsed content and versioned JSON/Parquet results.	Required. Standard bucket mk-lapi; use the new target's namespace, bucket and authorized credentials.
OCIR repositories and image access	Store immutable application images and let OKE pull them.	Required. Private lapi-web and lapi-backend. Pilot also retains lapi-ssh for its model relay.
Public load balancer, DNS and HTTPS	Give users the application URL and terminate HTTPS through the web pod's Caddy sidecar.	Required for public access. Pilot flexible load balancer 10 Mbps min/max, ports 80/443; HTTPS state is persistent.
Kubernetes namespace, Secrets and ConfigMaps	Provide workload identity/access, DB wallet, encryption key, settings, image pulls and target routes.	Required. lapi-oci; exact protected-state and resource contracts are listed on page 6.

The pilot DB Free Tier flag does not make the whole environment free; the observed 1 ECPU allocation is not a new-instance sizing default. The Gemini path requires no OCI Generative AI service.

Model and operator prerequisites

The three inspected active datasets use Google Gemini. Arrange its provider account, endpoint, credentials and quota separately from OCI. Oracle native VECTOR stores embeddings; a model generates them. Confirm model availability and parameters for the new provider account.

Role	Purpose	Observed model and required setup
Embedding model	Create document/query vectors for semantic retrieval and indexing.	Required for knowledge indexing/search. Saved model gemini-embedding-2 via Google Gemini API; a valid embedding registration, credentials and dataset mapping are needed.
Chat model	Generate evidence-based Unified Chat answers and citations after retrieval/reranking.	Required for chat. Saved gemini-3.5-flash-lite; register the Chat role and map it to the dataset.
Extraction and OCR model	Return structured extracted fields; read scanned/image content when OCR is needed.	Required for extraction and relevant scans. Saved gemini-3.5-flash-lite in the Extraction role; dataset-aware OCR uses that mapped registration.
CPU reranker	Rank retrieved evidence before chat/extraction generation.	Required for the production workflow. cross-encoder/ms-marco-MiniLM-L6-v2; pinned ONNX and tokenizer assets at /data/models/reranker inside OCI API/worker state.
Laptop Ollama alternative	Serve local embedding/generation models when deliberately selected instead of the cloud path.	Optional. Retained Qwen3 chat/extraction and Nomic embeddings; Qwen vision is an alternative OCR setting. Requires matching weights, powered-on laptop, Docker/Ollama and working relay. Not required by Gemini-mapped datasets.

Release computer and protected runtime state

Prerequisite	Purpose and when needed
Git and Python; chosen terminal	Windows: PowerShell 5.1+, Python 3.10+ (3.12 recommended). OCI Cloud Shell: Bash and Python 3.12. DB object work needs the pinned Oracle driver.
Image builder, kubectl and OCI auth	App releases use Docker on Windows or Podman in OCI Cloud Shell. kubectl and the kubeconfig authentication must work from that release environment. See pages 16-17 for Cloud Shell limits.
Workspace state, wallet and keys	Keep saved target DB/storage settings and encrypted secrets usable after restart. Use a new stable workspace key for a new installation; preserve that key on later releases.
Network and model access	Allow target access to DB/wallet service, Object Storage, OCIR and required model endpoint. Check provider quota and reranker integrity; full inference requires an authorized test.

The pinned reranker revision is 233902d25c440f23af6f7d6e94d2946bac0bee0a. The checked-in reranker module verifies model/tokenizer integrity; the release command does not download or replace these files. No dedicated GPU, Qdrant or separate vector database is part of the observed OCI/Gemini design.

A Provision networking and OKE

How our pilot was provisioned. The saved provisioning record confirms VCN, Internet Gateway, route table, three security lists/subnets, a Basic OKE cluster and the A1 node pool were created through the historical Python/OCI SDK helper. It recorded request IDs and retry tokens, checked Mumbai/home-region and capacity assumptions, and stopped rather than selecting a paid fallback. The pilot provisioning helpers have been retired from the active deployment folder; they are not a supported installer.

That historical workflow was specific to the pilot and its private state. The steps below explain equivalent destination preparation in OCI Console. They are separate provisioning actions, requiring the platform owner's target, sizing and configuration approval.

1 Select the destination and permissions

Record the tenancy, compartment, region, authorized user/profile and limits. Have the OCI administrator grant the approved permissions for networking, OKE/Compute, volumes, registry and bucket access, plus Kubernetes access. Check capacity and estimated charges. Use new names/IDs for a new environment and record every created resource.

2 Create the VCN and subnets

In Networking, create a VCN with non-overlapping address ranges. The pilot uses 10.42.0.0/16 with API 10.42.0.0/24, worker 10.42.1.0/24 and load-balancer 10.42.2.0/24 subnets. Flannel pod/service ranges are 10.244.0.0/16 and 10.96.0.0/16. Its public subnets use an Internet Gateway and default route. For a new private-node design, arrange the appropriate NAT/service gateways and management route. [1]

Create the destination security lists or NSGs for OKE control-plane/worker communication, approved management access, app ports 80/443 and required outbound traffic. In the pilot, management 6443/22/32022 is restricted to the recorded operator /32; 32022 belongs only to its optional SSH model relay. Do not expose the Oracle database or Ollama model port as public application ports.

3 Create the cluster and node pool

In Kubernetes Clusters, choose Create cluster and Custom create, select the prepared network, and use Basic when the chosen features permit it. The pilot used Flannel, a public API endpoint, and one managed ARM64 VM.Standard.A1.Flex worker with 2 OCPUs, 12 GB RAM and a 50 GB boot disk. Select an available supported Kubernetes version and matching OKE node image for the destination; v1.36.4 is the observed pilot version, not an availability promise. [2]

4 Obtain access and confirm the result

Wait for the cluster and worker to become available/Ready. Use Access cluster > Local Access to generate a destination-specific kubeconfig with the authorized OCI identity. Private APIs require the approved network/bastion path. Record context, server and architecture, then inspect nodes and storage classes. Do not reuse the pilot's private kubeconfig helper/state for a new tenancy. [3]

[1] Oracle OKE networking | [2] Oracle Custom create | [3] Cluster access

B Provision database storage and registry

Our pilot example: Lapi connects to the existing Autonomous AI Database lapiora and Standard Object Storage bucket mk-lapi. The OKE provisioner does not create either service. Their connection/configuration was verified; this handover does not claim a recorded original Console creation sequence for them.

1 Create the Autonomous AI Database

In Oracle AI Database > Autonomous AI Database, select the destination region/compartment and Create Autonomous AI Database. Enter new display/database names, select the approved workload, and choose Oracle AI Database 26ai with native VECTOR/BOOLEAN and compatible collation support. Set the permitted compute/storage, scaling, network access and private ADMIN password; review charges and create. Wait for AVAILABLE. The pilot observed 1 ECPU/20 GB with the Free Tier flag; a new target must use the options actually available to it. [4]

Choose the network path permitted by the destination policy and ensure both the operator and OKE can reach it. Under Database connection, download that database's wallet into a private directory. Record its TNS alias and full SERVICE_NAME; keep the wallet password private. Retain mTLS for a wallet-based installation. [5]

2 Create the application objects

Use pages 11-13 with the new wallet, owner, admin identity and service. For a database-only run, set DB_Object Y and both app flags N, and review the missing schema/grants/quota and ordered object plan before PUBLISH. This creates only supported missing objects in the already provisioned database. An incompatible database capability or existing schema must be resolved before proceeding; the release tool does not change database-wide settings.

3 Create the document bucket

In Object Storage, choose the destination compartment and Create bucket. Use a target-specific name, Standard tier and the approved access/versioning choices. Record region, namespace and bucket, then grant the application identity the needed object operations. The inspected pilot bucket had ObjectRead public access and versioning Disabled; those are observations, not required application settings. Choose the new target's approved policy and do not change live to copy this example. [6]

4 Prepare image repositories and credentials

In Container Registry, create private lapi-web and lapi-backend repositories in the approved destination compartment. Retain a separate relay image only if that architecture is selected. Record the registry domain and tenancy namespace; the pilot domain is bom.ocir.io. Configure authorized operator push access and Kubernetes pull access. Docker login on the laptop does not create the namespace's image-pull Secret. [7]

[4] Create Autonomous AI Database | [5] Download wallet
[6] Create Object Storage bucket | [7] Private image pulls

C Prepare the initial application installation

A new cluster needs a reviewed destination-specific first-install package. The code-release command updates existing workloads. The historical pilot renderer has been retired from the active deployment folder and is not an installer. Its private-state, hostname and relay assumptions are not portable. The platform/application owner must prepare reviewed manifests, OCI configuration and reranker files before the release workflow.

Initial installation item	What the platform/application owner prepares
Namespace and workloads	lapi-oci; lapi-web with web and https; lapi-api-worker with api and worker using the same compatible backend image. The pilot also has a model-relay sidecar. Use target-platform image digests and the reviewed single-replica Recreate contract.
Persistent volumes	application-state and https-state claims with oci-bv; pilot 50 Gi each, ReadWriteOnce. API/worker share /data. Arrange UID/GID 10001 access and compatible scheduling. CSI provisions backing OCI block volumes when claims bind. [8]
Secrets and mounts	ocir-pull; lapi-app; oci-credentials; database-wallet mounted read-only at /run/lapi-db/wallet. ssh-relay only for an approved relay design. Supply secret payloads privately, not in Git or command arguments.
Protected saved state	Destination configuration/database.json with active_provider=oci, active_connection and a valid encrypted secret_ref; configuration/settings.json with storage_provider=oci and target bucket route. Use independent destination credentials and a new stable workspace encryption key; preserve its relationship to encrypted state.
Runtime and model files	lapi-config, lapi-runtime-config and OCI runtime profile; CPU reranker assets in /data/models/reranker. Do not use local Compose/container_prepare as an OCI initializer: that separate path expects other local services.
Web routing and certificates	Internal api ClusterIP on 8000; web on 3000; Caddy reverse proxy and persistent certificate state; lapi-public LoadBalancer exposing 80/443. The reviewed pilot manifest requests a flexible 10/10 Mbps load balancer.

How the URL appears: the initial LoadBalancer Service obtains its public address from OCI. Record it, configure the approved DNS hostname and Caddy HTTPS host, then verify certificates and reachability. The pilot uses an lapi.<public-IP>.sslip.io hostname. This first-install DNS/TLS configuration is outside deploy.ps1; later code releases discover and preserve the existing URL.

The platform owner installs only the reviewed package for the approved new destination, then checks bound storage, correct mounts, images, configuration and readiness. No general unattended empty-tenancy or protected-state bootstrap installer is provided by the current release tools. If that package/state is missing, stop and prepare it before the release command.

[8] Oracle CSI block-volume claims

D Prepare models and complete the handoff

1 Install the pinned CPU reranker assets once

The deployed backend runs ONNX reranking on its CPU and expects onnx/model.onnx and tokenizer.json under /data/models/reranker. The maintained setup helper downloads the fixed model revision and verifies hashes. For this separate initial asset-staging task, use a local checkout and supported Python environment; create .venv with py -3.12 -m venv .venv only if it is missing. On an authorized preparation computer, install the shared requirements and stage the assets:

```
.\.venv\Scripts\python.exe -m pip install `
  -r lapi_backend\requirements.txt
.\.venv\Scripts\python.exe `
  .\lapi_local_use\lapi_scripts\setup-reranker.py `
  --directory C:\lapi-target\reranker
```

The platform owner transfers those verified files into the **new destination's** application-state volume at models/reranker, with the required permissions, before startup. The staging command alone does not install files in OCI. Model downloads occur only during this initial setup; code releases preserve the installed assets.

2 Register and map the chosen cloud models

Arrange a permitted Google Gemini endpoint, private API credential and sufficient quota for the new destination. In the initialized Lapi workspace, register and test the Embedding, Chat and Extraction roles, then map the dataset to them. The pilot's saved examples are gemini-embedding-2 and gemini-3.5-flash-lite; confirm that the actual provider account supports the chosen identifiers and parameters. OCR for the inspected Gemini datasets uses the mapped Extraction registration, not the retained local OCR default.

Use a compatible embedding configuration and dimensions for the destination's native vectors; changing an established embedding mapping can require reindexing and is not a code-release action. Complete authorized workspace administrator/bootstrap setup if the new workspace is empty. New model registration, dataset mapping, key setup and any validation inference belong to this initial installation scope.

3 Keep local models optional

Only if the selected design uses laptop Ollama, prepare its required model weights and approved SSH relay and verify cloud-to-laptop reachability. The laptop, Docker/Ollama and relay must remain online for those requests. No laptop model is required by the observed Gemini-mapped datasets. Local container health is not proof of a working relay or model inference.

4 Hand the prepared destination to the release operator

Record the resource inventory, context/server, namespace, registry, architecture, DB owner/full service/wallet alias, bucket route, HTTPS URL and model availability. Verify the app's protected saved state, bound volumes, reranker integrity and readiness. A harmless upload/index/extract/cited-chat acceptance test requires a test dataset and authorized writes/provider calls; record what actually passed.

Fill the non-secret JSON on page 11 using this target. Complete Windows setup on page 10 and the page 15 preflight; after it passes, use the AI or terminal route with the exact component flags and final release approval. Provisioning is now complete; the subsequent release changes only the reviewed missing DB objects/schema and selected application images.

This provisioning reference was checked against retained pilot source/creation records and linked Oracle documentation on 16 October 2026. No resources, configuration, database objects, model registrations or images were changed to prepare it; no new-tenancy installation was rehearsed.

2 Code deployment

Confirm the destination is prepared

For another tenancy or database, obtain that destination's access and configuration. Do not reuse the original live wallet, owner, kubeconfig or endpoint. Tenant details alone are insufficient. Ask the platform/application owner to confirm the items below before publication.

Prerequisite	Required handoff
Oracle database	An existing Autonomous AI Database supporting the maintained native VECTOR, SQL BOOLEAN and collation definitions; reachable with the target wallet and authorized account.
Application workloads	Existing Lapi Deployments and container names, namespace, Services, persistent claims and shared API/worker state. Ready nodes alone are insufficient.
Saved configuration	Target DB connection and wallet, Object Storage route, encryption key, credentials and runtime settings already installed. Existing model registrations and CPU reranker assets must be available.
Access and registry	Destination kubeconfig/context and its authentication executable; authorized Kubernetes access; existing registry repositories, push login and cluster pull access.
Public access	Prepared HTTPS routing and hostname. The tool can discover an existing URL; it does not create DNS, certificates, Ingress or a load balancer.

For a missing application schema, the release plan may label app readiness as conditional. After approved DDL, the saved connection and application must pass checks **before images are published**. Creating tables alone does not seed application users, models, documents or configuration.

The full OCI resource and model prerequisite tables, with each item's purpose and the observed pilot example, are on pages 2-3.

Prepare the release computer

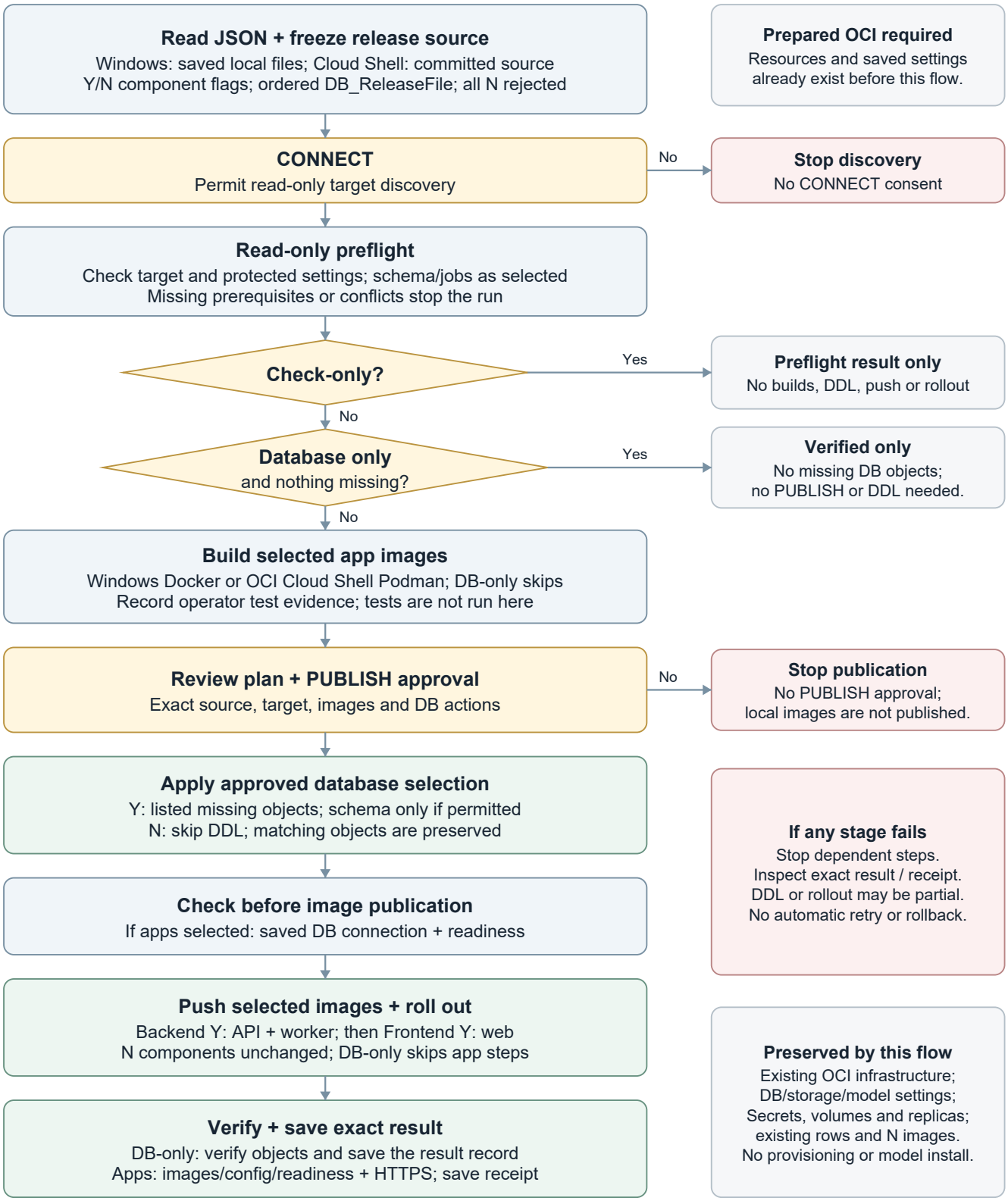
For Windows, have PowerShell 5.1+, a local Git checkout, internet access and App Installer/WinGet available. **Setup-Lapi.ps1 installs missing supported tools and verifies readiness**; see page 10. Your destination files, access and prepared OCI resources must be supplied. For builds inside OCI without Docker Desktop, use Cloud Shell on pages 16-17; that route continues to use committed source.

Keep the wallet, kubeconfig, private keys and target JSON outside Git. The current Mumbai pilot kubeconfig uses a private local authentication helper/state; it is not portable to another tenancy. Obtain a destination-specific kubeconfig from its owner.

Database-only selection: no Docker, Kubernetes, registry or application URL is required. Use DB_Object Y, DeployFrontend N and DeployBackend N. Missing prerequisites stop the release; resolve them outside this code/object-only scope.

Deployment script workflow

First complete Windows prerequisite setup on page 10. Both terminal routes read the target JSON: Windows freezes saved local files and builds with Docker; Cloud Shell exports a commit and uses Podman. Follow the release path downward. Prepared OCI resources are required.



Failure handling applies to every stage. Inspect the exact result/receipt before an approved retry; successful preflight does not authorize publication.

Windows prerequisites and automatic setup

Use an already-open **Windows PowerShell 5.1+** window. Have a local Lapi Git checkout, internet access, Microsoft App Installer/WinGet and permission to complete installer/UAC prompts. Software installation cannot create OCI access or the prepared installation on page 8.

Have the destination handoff ready

Provide a filled private JSON, usable destination kubeconfig/context and its authorized authentication, registry credentials, and truthful validation evidence for your saved files. For DB_Object Y, provide the destination wallet ZIP, owner/admin identity, alias and full service. For app builds, allow at least **10 GiB free on the source/work drive**; Docker image storage may need more.

Tool or dependency	What normal setup does
Git and Python	Reuses working tools. Installs missing Git/Python 3.12 with WinGet; creates a missing project .venv. An existing broken or unsupported .venv stops for repair.
kubectl and Docker Desktop	For application releases, installs missing clients, starts Docker if needed and verifies Linux containers and builder support for the target architecture.
OCI CLI	Installs an isolated tool environment only if kubeconfig authentication needs OCI CLI and it is missing. Existing OCI credentials/profile are still required.
Oracle driver	For DB_Object Y only: installs/verifies the maintained pinned Oracle requirement. DB-only releases skip Docker and Kubernetes.
WSL and restarts	Can request missing WSL setup when supported and run with administrator access. Complete prompts, restart if requested and rerun setup; no automatic reboot.

1 Run prerequisite setup

From your checkout's lapi_deploy folder, use the default adjacent private JSON:

```
.\Setup-Lapi.ps1
```

Type **CONNECT** when the visible prompt asks for read-only target access. Complete installer, Docker first-run and private login prompts. Continue only after **LAPI_PREREQUISITES_READY**; press Enter at the finish prompt. A failure stops setup and prints its saved result path.

Setup checks local tools, the saved-source snapshot, target/schema/configuration, builder architecture and matching current-image metadata. It does not build/push images, apply DDL or change workloads. Push permission, actual builds and future rollout success are not guaranteed by readiness.

Checks only: add -CheckOnly to Setup-Lapi.ps1. It still needs CONNECT and performs remote read-only checks, but does not install software, create .venv, start Docker or log into the registry. Results are under ignored lapi_runtime/deployment-setup/.

The full manual installation commands are in lapi_deploy/README.md; normal setup handles missing supported tools automatically. Page 11 prepares another destination JSON; page 15 gives the deployment command.

Prepare the source and destination JSON

Reuse your own Lapi checkout without overwriting pending work. If you do not have a checkout, obtain one with Git history from the repository owner, or clone the branch with Git installed. Choose that computer's lapi_deploy folder before running the scripts.

Create a private target file once

From lapi_deploy, copy the clean template only if the destination file is missing:

```
New-Item -ItemType Directory -Path C:\lapi-target -Force
if (-not (Test-Path C:\lapi-target\colleague-oci.json)) {
  Copy-Item .\templates\deployment-inputs.example.json `
    C:\lapi-target\colleague-oci.json
}
notepad C:\lapi-target\colleague-oci.json
```

For a different prepared tenancy/region/cluster, use its own values and authorized access. Pass the **same JSON to both scripts with -Inputs**. Without -Inputs, the default is lapi-oci-inputs.json beside the scripts. Missing JSON is copied from the template by setup, which then stops for you to fill it.

JSON fields	What to supply
SourcePath / Ref	Local checkout path; blank SourcePath uses this checkout. Windows builds saved files by default and ignores Ref. -CommittedSource selects Ref; Cloud Shell always uses a commit.
Kubeconfig / Context / Namespace	Destination-specific file, usable authentication/profile, exact context and existing Lapi namespace. Another laptop's private helper paths may not work.
Registry / RegistryNamespace	Destination OCIR hostname without https:// and tenancy registry namespace. Private Docker login does not create cluster image-pull credentials.
Platform / AppUrl	linux/arm64 or linux/amd64 matching nodes, and existing HTTPS URL; supported blank values are discovered during CONNECT preflight.
WalletPath / DatabaseOwner / DBAdminUser	For DB_Object Y: destination wallet ZIP, application owner and authorized inspection/admin account.
ServiceAlias / ExpectedService	Destination wallet TNS alias and full database SERVICE_NAME. They must identify the intended database.
ValidationReference	Truthful test/CI evidence for the saved files or selected committed source. The scripts do not run the regression suite.

Use the Y/N selectors on page 12. Keep passwords, tokens, private keys and wallet contents out of JSON. Use a separate private file per destination; no global destination is remembered.

Select exactly what to deploy

Use the following selection fields inside the full target JSON. The remaining destination fields come from the template on page 11. Use quoted Y or N values.

```
{
  "DB_Object": "Y",
  "DB_ReleaseFile": "lapi_database/release.json",
  "DeployFrontend": "Y",
  "DeployBackend": "Y"
}
```

Release selection	DB_Object	Frontend	Backend
All three	Y	Y	Y
Database only	Y	N	N
Frontend only	N	Y	N
Backend API + worker only	N	N	Y
Both code components; no DB DDL	N	Y	Y

Backend includes both API and worker. Unselected images are not built, pushed or changed. All three N is rejected. With DB_Object N, a backend release still checks schema compatibility and active jobs read-only; frontend-only with DB_Object N makes no Oracle queries.

The database release file controls the script list

`lapi_database/release.json` contains **CreateSchemaIfMissing: "Y"** and an ordered **Scripts** array. The supplied manifest lists 13 maintained SQL files under lapi_database/lapi_tables, containing 13 table and 20 explicit index definitions. Only these listed files participate; the tool does not scan a folder for more scripts.

On Windows, repository manifest/SQL paths use the saved local snapshot by default; save edits before running. With -CommittedSource or Cloud Shell, they use the selected committed source. An external absolute manifest is frozen in the private record; its SQL paths still resolve inside the selected source snapshot. Only the reviewed release-file objects can be applied after PUBLISH.

This release supports the maintained CREATE TABLE/INDEX definitions. Arbitrary migrations, stored-program releases, ALTER/DROP statements and DML scripts require separate implementation and review. Do not add them expecting a general SQL runner.

Review database and runtime behavior

When DB_Object is Y

Read-only preflight compares the listed object definitions against the intended owner and full service. Matching tables and indexes are skipped. Missing supported objects are created in manifest order only after approval. Incompatible definitions stop the release before DDL; row counts or table counts alone are not a schema comparison.

If the application user/schema is missing and CreateSchemaIfMissing is Y, the plan shows the new account, CREATE SESSION and CREATE TABLE grants, existing DATA tablespace and default 256 MB quota. Optional SchemaTablespace and SchemaQuotaMB select other reviewed existing-space settings. Existing accounts, passwords, grants and quotas are preserved. No physical database or tablespace is created.

Wallet, admin and application-owner passwords are entered privately when requested in the interactive terminal. A new schema password is confirmed privately. They are not saved in JSON, argv, generated SQL or release records. The database applies its password policy.

No existing application rows are imported or changed by this object release. A missing schema is an empty application schema; it does not migrate the original workspace. Database-only with every listed object already matching finishes as verified-only without PUBLISH because there is no change to apply.

Preserve the maintained build profile

Area	Maintained release profile
Backend base	Python 3.12.15 with the immutable base digest matched to the observed live ARM64 image.
Frontend base	Node.js 22.23.3 with the immutable live-matching base digest; production npm ci uses package-lock.json.
Backend dependencies	Pinned requirements and lockfiles from the selected source; Oracle, OCI and native AWS/Google SDK bundles enabled. Local in-process embeddings disabled, matching live.
Model and application state	Existing provider mappings, remote Ollama endpoints, reranker files, encryption keys, runtime settings and storage routes preserved.

Do not change Docker build flags, base digests, dependencies or model configuration during a release to work around a failed preflight. Keep required providers and any deliberately selected laptop model relay available. A powered-off laptop cannot serve its Ollama model; Gemini-mapped datasets use their existing cloud provider.

The Windows colleague workflow is Setup-Lapi.ps1 then Deploy-Lapi.ps1, which calls deploy.ps1; OCI Cloud Shell uses cloud_shell/deploy-cloud-shell.sh. Shared implementation is under lapi_deploy/scripts/. Do not bypass approvals by running an underlying publish helper directly.

Deploy using the AI skill

Give the AI your source location, saved-file or committed-source choice, target JSON and component selection. Use the repository skill at `.agents/skills/lapi-deploy/SKILL.md`. If the tool recognizes installed skills, invoke **$lapi-deploy**; otherwise ask it to read and follow that file.

Copy this request to the AI

Use $lapi-deploy and follow .agents/skills/lapi-deploy/SKILL.md in my Lapi checkout. Deploy the saved local application source using C:\lapi-target\target.json; do not require a Git commit for this Windows run. Use its DB_Object, DB_ReleaseFile, DeployFrontend and DeployBackend selections. Preserve all existing OCI infrastructure, application configuration, model registrations and data. Do not provision infrastructure or change settings. Ask me for missing inputs; if unknown, request permission for bounded read-only discovery. Before publishing, show the complete verified release plan and wait for my explicit approval.

The AI must complete these stages

1. Resolve inputs locally. Check source, destination file and selected components. Ask about unknown values. Sharing a tenancy ID does not select a unique cluster or database, and the AI must not guess the target.

2. Obtain connection permission. Confirm the account/profile, tenancy, region and discovery scope before OCI, Kubernetes, Oracle or public-target checks. Existing consent for the same release scope can be reused. Discovery is read-only and does not permit publication.

3. Prepare a concrete release plan. Verify target identity and prerequisites; complete applicable source checks and local image builds. Show the source mode, snapshot fingerprint/manifest and base commit (or exact committed Ref), cluster/context/server, namespace, architecture, registry, URL, current and proposed images, DB owner/full service, script hashes, missing objects, any new schema/grants/quota, active jobs, preserved configuration fingerprints, evidence, unrun checks and expected restart window.

4. Obtain final approval. Wait for explicit approval of that exact plan before DDL, image pushes or workload updates. Credential access, read-only consent and successful preflight are not publication approval. Recheck changed source/target details and obtain amended approval if the plan changes.

5. Execute and report. Apply only approved missing DB objects and selected images, verify configuration and readiness, and return source/target, DB outcome, image digests, HTTPS URL, exact receipt and any limitations. Stop dependent work on failure; do not automatically retry or roll back.

Use a private credential mechanism or the operator terminal for secrets. Do not paste passwords or tokens into the AI request. An unknown application URL may be discovered from existing routing after permission; the skill must stop if routing or other initial setup is missing.

Windows: build and deploy saved files

Complete prerequisites and setup on pages 8-11. Work in your checkout's **lapi_deploy** folder. For the adjacent private lapi-oci-inputs.json, your colleague uses two scripts in order:

```
.\Setup-Lapi.ps1
# After LAPI_PREREQUISITES_READY:
.\Deploy-Lapi.ps1
```

Each time you save code and want a release

Save application files, then run **.\Deploy-Lapi.ps1**. It builds the selected components from a fresh local snapshot. **No Git commit or Ref update is needed.** New non-ignored source files and saved deletions are included. Edits after snapshot creation wait for the next invocation. Saving a file alone does not launch a deployment.

Known private files, Git-ignored new files, runtime data and generated output are excluded. Every release runs Docker build for selected app images; unchanged layers may use cache. The snapshot is frozen under the run directory and source-manifest.json lists file hashes. In working-tree mode, source_commit records only the base Git commit, not the exact file contents.

Approvals and execution

1 CONNECT: review the destination and permit read-only preflight. Tests are not run here; supply truthful ValidationReference evidence for the files being released.

2 Build: wait for the selected local Docker builds. Backend includes API and worker. DB-only skips image builds.

3 PUBLISH: review the saved plan: source_mode, snapshot fingerprint, destination, images and database actions. Type PUBLISH only to approve that exact plan.

4 Release and verify: recheck source/target/images; apply approved DB actions, push selected images, update existing workloads and verify the result. Preserve the printed Application, Receipt and Record paths. No automatic retry or rollback.

Another prepared OCI destination

```
.\Setup-Lapi.ps1 -Inputs C:\lapi-target\colleague-oci.json
.\Deploy-Lapi.ps1 -Inputs C:\lapi-target\colleague-oci.json
```

Run the second command only after setup readiness. Setup prints the exact matching deploy command. Use the same destination JSON in both commands; credentials and infrastructure must belong to that target.

Other Windows invocation options

Direct main script: .\deploy.ps1 uses the same default saved-file behavior. **Target check:** .\Check-Lapi.ps1, or .\deploy.ps1 -CheckOnly. **Pinned commit:** add -CommittedSource consistently to setup and deploy; only then does JSON Ref select committed source. All accept -Inputs for another target.

Selected Recreate updates restart pods and can briefly interrupt service. Configuration, model mappings and stored data are preserved. Wait for the completion message and exit code 0; setup readiness alone is not a deployment.

OCI Cloud Shell: prepare once

Use this route to build inside OCI without Docker Desktop. Sign in privately to the intended tenancy, choose its region, then open Developer tools > Cloud Shell. Verify the account before connecting. The supplied command does not create infrastructure or enable OCI DevOps managed builds.

Cloud Shell compute is free within the tenancy monthly allowance. The Mumbai account showed 160 hours available before this session. The runner checks remaining allowance and stops below two hours; there is no paid fallback. Existing application hosting and registry storage have separate limits and pricing. Confirm registry capacity before a push under a zero-additional-cost requirement.

Check	Requirement and observed session
Architecture	Native architecture must match OKE nodes. Observed: ARM64, 2 CPUs, about 12 GB RAM; Podman 4.9.4.
Existing disk	Observed: 5 GB home and about 4.6 GB free in /tmp. Before builds, at least 3.5 GiB must be free on each selected component filesystem.
Tools	Git, OCI CLI, kubectl and Podman are supplied. Use Python 3.12 directly; no global runtime change is needed.

Obtain source and the isolated Oracle tool environment

For an existing checkout, reuse it and fetch the reviewed commit without overwriting pending work. For a fresh checkout:

```
git clone --depth 1 --branch lapi_sprint1 \
  https://github.com/Maniatramco/Lapi.git "$HOME/Lapi"
cd "$HOME/Lapi"
mkdir -p "$HOME/lapi-private"
python3.12 -m venv "$HOME/lapi-private/tools"
export LAPI_PYTHON="$HOME/lapi-private/tools/bin/python"
"$LAPI_PYTHON" -m pip install -r lapi_backend/requirements-oracle.txt
cp lapi_deploy/templates/deployment-inputs.example.json \
  "$HOME/lapi-private/target.json"
```

Prepare client access to the existing cluster

```
read -r -p "Existing cluster OCID: " task_cluster
oci ce cluster create-kubeconfig --cluster-id "$task_cluster" \
  --file "$HOME/lapi-private/kubeconfig.yaml" \
  --region "$OCI_REGION" --token-version 2.0.0 \
  --kube-endpoint PUBLIC_ENDPOINT
```

This writes a client kubeconfig; it does not change the cluster. Use an unused filename rather than overwrite an existing client file. For DB_Object Y, privately upload the approved destination wallet ZIP into lapi-private. Do not copy passwords, local private keys or saved credentials through chat. If the agent uploads a wallet, approve that specific file and OCI destination first.

Oracle reference: Cloud Shell tools, storage and free usage limits

OCI Cloud Shell: check and deploy

Current Mumbai status, 16 October: the existing OKE API allowlist blocks Cloud Shell public access. The user chose to preserve all network settings. Full deployment from this session is therefore blocked; no firewall exception or production release was made.

Edit target.json using Cloud Shell paths and the same Y/N selections on page 12. Keep credentials outside JSON. Windows file paths do not work here.

Input	Cloud Shell value
SourcePath / Ref	Full path to this checkout and exact tested commit containing the Cloud Shell route. Its controller files must match the selected source.
Kubeconfig / Context	Full path to lapi-private/kubeconfig.yaml. Blank Context selects its sole context; multiple contexts require an explicit choice.
WalletPath	Full path to the privately uploaded destination wallet ZIP when DB_Object is Y.
Other destination fields	Use the verified namespace, registry, registry namespace, architecture, HTTPS URL, DB owner, admin, wallet alias and full database service.

Authenticate the registry privately

```
read -r -p "Destination OCIR host: " task_registry
podman login "$task_registry"
```

Use your existing approved OCIR username and token in the private interactive prompts. This logs in the client and does not publish images. The agent must not transfer saved credentials without permission. DB-only does not require registry login.

Read-only preflight, then the guarded release

```
cd "$HOME/Lapi"
export LAPI_PYTHON="$HOME/lapi-private/tools/bin/python"
bash ./lapi_deploy/cloud_shell/deploy-cloud-shell.sh \
  --inputs "$HOME/lapi-private/target.json" --check-only
# After reviewing a successful preflight:
bash ./lapi_deploy/cloud_shell/deploy-cloud-shell.sh \
  --inputs "$HOME/lapi-private/target.json"
```

Type CONNECT only for the intended read-only target checks. Supply DB passwords privately if selected. Check-only performs no builds or publication. The release command builds selected images in isolated stores, shows the complete plan and waits for PUBLISH. After approval it applies reviewed missing DB objects, verifies readiness, pushes selected images and updates only their existing containers by immutable digest.

Retain the exact printed result and receipt paths. Recreate restarts can briefly interrupt service. Legacy DatabaseAction/SQLcl is not supported by this Cloud Shell route; use DB_Object and DB_ReleaseFile. Provider inference and upload/extraction tests are not run.

Build stores can consume most of Cloud Shell storage. Keep a stopped run for inspection. After a completed run, the operator may discard only its recorded isolated stores when no longer needed; preserve receipts. Never globally prune Podman, delete another run or add paid storage to bypass a space failure. /tmp is temporary and images there must be rebuilt after the session is recycled.

Verify and hand over the result

Retain the private run directory under ignored `lapi_deploy/deployment-records/`. Use the exact paths printed by the run, rather than selecting a receipt by the newest filename. Keep destination details and records outside public Git history.

Record	What to confirm
Source and destination	Source mode and snapshot fingerprint/manifest; base commit for saved files or exact committed Ref. Approved target/context/namespace, DB identity and flags.
Database result	Created or skipped objects; new schema only if approved; final compatibility check. Database-only completion does not claim an application rollout.
Application result	Previous/new image digests, selected workload readiness, unchanged unselected images and protected configuration, existing HTTPS URL/status.
Functional acceptance	Check sign-in and read-only screens. A new upload, extraction or cited-chat test needs a prepared test workspace and explicit authorization for writes/inference.

If the command stops

Stop and inspect the recorded phase, completed DB objects and actual image versions. Oracle DDL can commit partially. A failed frontend rollout may follow a successful backend rollout. Do not blindly rerun, replay the fresh installer, run rollout undo, change settings, or claim a partial release succeeded. Recovery or restoration of previous compatible images requires its own approved action; no automatic rollback is provided.

Validation recorded on 16 October 2026

Historical baseline: 1,047 backend tests (5 skipped), 167 frontend unit tests, frontend type checks and 69 deployment tests passed. ARM64 images built; frontend checks returned HTTP 200; 105 backend package versions matched live. Oracle comparison matched 13 tables, 165 columns, 49 foreign keys and 20 indexes.

Live health remained unchanged during that review. No live provider inference, upload/extraction/chat write workflow, new-tenancy installation or recovery drill was performed. These results support that source snapshot; a different target and changed code still need their own verification. A branch push is not deployment and is not a new CI run.

Historical Cloud Shell validation: 15 route checks (85 deployment checks total); both ARM64 Podman images built, SDK checks passed and isolated frontend/assets returned HTTP 200. That session was blocked by API access; it made no production release and enabled no paid runner.

Verified application baseline: `3ae03a5f40fa24bd7c05f84892a9d73efddc677d`. Later commits add handover, folder organization and the Cloud Shell route. The original immutable tag lapi-freeze-2026-10-15 remains unchanged.

Current Windows update: 118 deployment checks ran (1 skipped), covering setup, visible prompts, saved-file snapshots and approval guards. A 253-file local snapshot was verified. No new image build, OCI connection or rollout was performed for this documentation update.

Maintained source and reference

Windows workflow: Setup-Lapi.ps1 then Deploy-Lapi.ps1; full commands
Input template: deployment-inputs.example.json
Object list: lapi_database/release.json
AI workflow: lapi-deploy skill on lapi_sprint1

Architecture and the approved Mural diagram are maintained separately in Lapi_Architecture_Handover.docx and its matching PDF. Pages 2-7 add the resource/model prerequisites, pilot provisioning example and first-install handoff. This document replaces the separate database, backend and Kubernetes deployment guides.